

Name: Saqlain Hashim shah

Sap: 57139

Subject: SQE

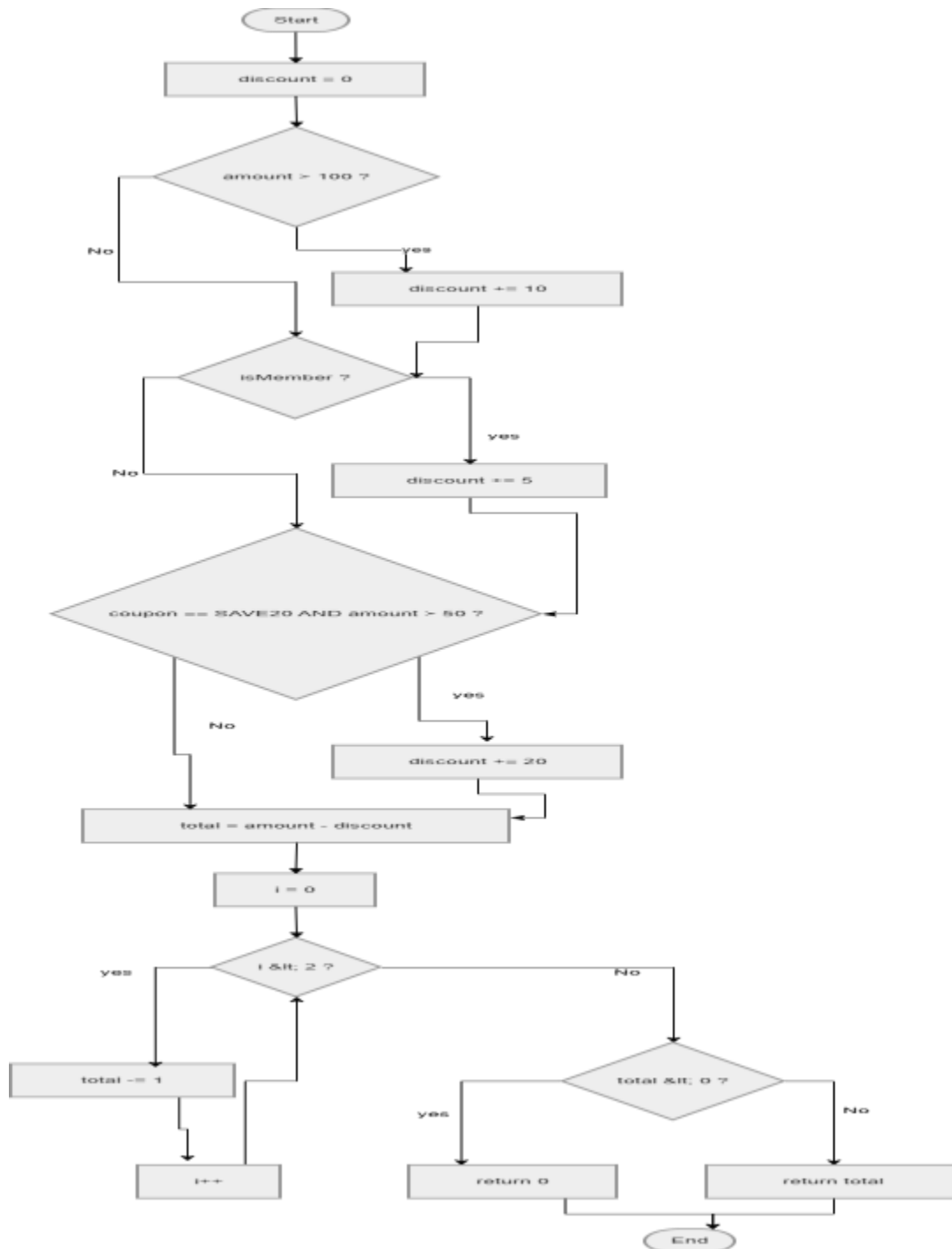
Teacher: Dr.M.Fawad

Lab No 3



Part 1

Control Flow Graph



Part 2

Basis Path Testing

Step 1: Cyclomatic Complexity

Formula:

$$V(G) = \text{Number of Decision Nodes} + 1$$

Decision Nodes:

1. `amount > 100`
2. `isMember`
3. `coupon == "SAVE20" && amount > 50`
4. `i < 2`
5. `total < 0`

Therefore:

$$V(G) = 5 + 1 = 6$$

Step 2: Independent Paths

Path	Description
P1	No conditions true, loop executes
P2	Only <code>amount > 100</code> true
P3	Only <code>isMember</code> true
P4	Coupon condition true
P5	Multiple conditions true
P6	Total becomes negative

Step 3: Path Test Cases

Path	Input (amount, member, coupon)	Expected Output
P1	(40, false, "NONE")	38
P2	(120, false, "NONE")	108
P3	(80, true, "NONE")	73

Path	Input (amount, member, coupon)	Expected Output
P4	(60, false, "SAVE20")	38
P5	(150, true, "SAVE20")	113
P6	(10, false, "SAVE20")	0

Part 3

Condition Testing

Focus Condition

```
if (coupon == "SAVE20" && amount > 50)
```

Atomic Conditions

- C1: coupon == "SAVE20"
- C2: amount > 50

Condition Testing Table

Test	C1	C2	Input	Expected Result
T1	True	True	(60, false, "SAVE20")	Discount Applied
T2	True	False	(40, false, "SAVE20")	Not Applied
T3	False	True	(60, false, "NONE")	Not Applied
T4	False	False	(40, false, "NONE")	Not Applied

Part 4

Loop Testing

Loop Statement

```
while (i < 2)
```

Loop Test Cases

Case	Iterations	Strategy
L1	0	Force loop skip
L2	1	Partial execution
L3	2	Normal execution
L4	>2	Stress testing

Case	Modification	Expected Behavior
L1	Set <code>i = 2</code> initially	Loop skipped
L2	Change condition to <code>i < 1</code>	Executes once
L3	Original code	Executes twice
L4	Change condition to <code>i < 5</code>	Executes more than twice

Part 5

Data Flow Testing

Variables

- `discount`
- `total`
- `i`

Def-Use Table

Variable	Definition (Def)	Use
<code>discount</code>	<code>discount = 0</code>	<code>total = amount - discount</code>
<code>discount</code>	<code>discount += 10</code>	<code>total = amount - discount</code>
<code>discount</code>	<code>discount += 5</code>	<code>total = amount - discount</code>
<code>discount</code>	<code>discount += 20</code>	<code>total = amount - discount</code>
<code>total</code>	<code>total = amount - discount</code>	<code>if (total < 0)</code>
<code>total</code>	<code>total -= 1</code>	<code>if (total < 0)</code>
<code>i</code>	<code>i = 0</code>	<code>while (i < 2)</code>

Variable	Definition (Def)	Use
i	i++	while (i < 2)

Def-Use Test Cases

Test Case	Purpose
(40, false, "NONE")	Basic definition-use coverage
(120, false, "NONE")	Covers <code>discount += 10</code>
(80, true, "NONE")	Covers <code>discount += 5</code>
(60, false, "SAVE20")	Covers <code>discount += 20</code>
(150, true, "SAVE20")	Covers all discount definitions
(10, false, "SAVE20")	Covers negative total condition

Part 6

Integrated Final Test Suite

Test ID	Input	Path Coverage	Condition Coverage	Loop Coverage	Def-Use Coverage	Expected Output
TC1	(40, false, "NONE")	P1	T4	L3	Yes	38
TC2	(120, false, "NONE")	P2	T3	L3	Yes	108
TC3	(80, true, "NONE")	P3	T3	L3	Yes	73
TC4	(60, false, "SAVE20")	P4	T1	L3	Yes	38
TC5	(150, true, "SAVE20")	P5	T1	L3	Yes	113
TC6	(10, false, "SAVE20")	P6	T2	L3	Yes	0